R3.04 · BUT2 Informatique · IUT du Limousin · RETAULT Maël, MERCERON Emile, NIDOS Theo, ROUILLON Raphaël

Exposé R3.04 Qualité de code

Fiche résumé : Java moderne

Écrire un code Java plus robuste et concis

Pourquoi Java évolue ?

Depuis 2017, Java sort une version tous les 6 mois (une tous les 3 ans avant), pour livrer plus vite des nouveautés qui rendent le code plus sûr et plus court, et rester compétitif face à Kotlin, Scala ou C#. Les versions LTS (support long) sont Java 8, 11, 17 et 21. Fil rouge de l'exposé : un système de paiement en ligne (carte bancaire, virement, espèces) :

- Avant 2017 : une version environ tous les 3 ans, et un langage jugé verbeux (getters, setters, casts explicites partout).
- Depuis 2017 : une version en mars, une en septembre. Une nouveauté arrive d'abord en preview (version de test) pour recueillir les retours des développeurs, puis elle est finalisée.
- Objectif : rester compétitif face à Kotlin, Scala ou C#, plus concis et plus proches du style fonctionnel.
- Versions LTS (support long) : Java 8, 11, 17 et 21.

R3.04 · BUT2 Informatique · IUT du Limousin · RETAULT Maël, MERCERON Emile, NIDOS Theo, ROUILLON Raphaël

Les classes et interfaces scellées

Par le passé, il n'existait pas de type scellés; le problème ? Toutes les classes contenues dans un projet pouvaient implémenter cette interface, et justement ces interfaces étaient considérées comme étant trop ouvertes. Sans `sealed`, n'importe quelle classe peut implémenter une interface : un if/else if qui oublie un cas d'une classe compile quand même, et le bug n'apparaît qu'à l'exécution. Un type scellé liste lui-même ses sous-types avec permits, c'est à dire que l'on donne à l'interface une liste des seules classes pouvant l'implémenter, et cela permet au compilateur de connaître tous les cas possibles. Chaque sous-type doit être final, sealed ou bien non sealed.

CODE ASSOCIÉ

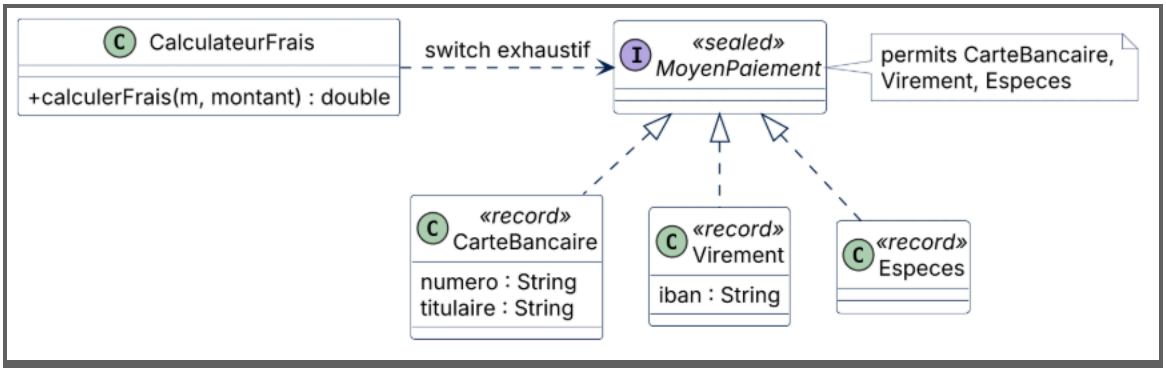
```
public sealed interface MoyenPaiement permits CarteBancaire, Especes, Virement, Bitcoin { }
```

Sealed & Record

Un record, est une classe conçue pour porter des données, le principe est que le compilateur génère les champs private, final, le constructeur, les accesseurs (numero(), sans get), equals, hashCode et toString. Ce dernier est immuable, et peut implémenter une interface :

```
public record Bitcoin(String adresse) implements MoyenPaiement {}
```

Lorsque l'on associe le type record à sealed, il permet de donner un type de données algébrique (ADT), comme dans les langages : Kotlin, Scala ou Rust.



Le compromis. La hiérarchie est fermée, donc si nous souhaitons ajouter une variante dans notre code, cela nous oblige à modifier les permits, afin d'y ajouter notre variante dans les types/objets autorisés à implémenter notre interface, mais également les `switch` qui en dépendent.

Le pattern matching

Le pattern matching, de Java 16 à 21, dans le principe, consiste simplement à vérifier qu'une valeur a une certaine forme tout en récupérant directement ses éléments, directement. Le concept est inspiré de langages fonctionnels comme Haskell ou Scala. Ici, pattern veut dire motif, pas patron de conception comme dans le GoF.

Avant Java 16, le `instanceof` imposait un test pour vérifier le type suivi d'un cast manuel et répétitif, sauf que ce code favorisait les sources d'erreurs. Depuis Java 16, on à donc la possibilité de déclarer directement la variable liée dans le if, et ainsi, elle est typé directement.

Avant Java16

```java
Animal animal = new Dog();

if (animal instanceof Dog) {
    Dog dog = (Dog) animal;
    dog.bark();  // Sûr d'appeler les méthodes spécifiques à Dog
}
```

```java
Dog dog = new Dog();
System.out.println(dog instanceof Dog);     // true
System.out.println(dog instanceof Animal);  // true
System.out.println(dog instanceof Object);  // true
```

Depuis Java16

```java
if (animal instanceof Dog dog) {
    dog.bark();  // 'dog' est déjà casté et prêt à être utilisé
}
```

Java 21 généralise cette logique au switch aussi, (déjà une expression depuis Java 14), c'est à dire que l'on élimine les répétitions en cascades de if/else if, on gère explicitement le cas om le type est null avec case null.

```
double frais = switch (m) {
      case CarteBancaire cb -> 1.5;
      case Virement v     -> 0.0;
      case Especes e      -> 0.0;
      // aucun default !
};
```

En implémentant ça à une hiérarchie scellée, le switch est dis exhaustif, autrement dit, le compilateur connaît tous les sous-types autorisés et vérifie qu'on a tout traité, donc on a plus la nécessité d'implémenter une case default. Si l'on ajoute un sous-type, et que l'on oublie d'implémenter le cas de notre nouveau sous-types dans notre switch, cela va cesser de compiler : **un oubli devient une erreur de compilation, plus un bug silencieux en production**

R3.04 · BUT2 Informatique · IUT du Limousin · RETAULT Maël, MERCERON Emile, NIDOS Theo, ROUILLON Raphaël

Lambdas, références de méthode et Streams

Les **lambdas** sont des fonctions anonymes. Les **références de méthode** offrent une syntaxe plus concise au code pour utiliser des méthodes ou des constructeurs déjà codés : statique (`Integer::parseInt`), sur un objet précis (`System.out::println`), sur n'importe quel objet du type (`String::length`) et constructeur (`ArrayList::new`).

- Style **impératif** : on explicite étape par étape le traitement, en contrôlant à la main l'itération et les états intermédiaires.
- Style **fonctionnel** : on exprime directement l'intention du résultat voulu, par exemple en combinant filtrage, transformation et agrégation.

Un Stream est un pipeline de traitement à usage unique, c'est à dire qu'il ne peut être parcouru et consommé qu'une seule fois, toute tentative de le réutiliser après une opération déjà passée, donc qu'il l'a déjà consommé, provoquera une erreur. Un Stream ne conserve, donc, aucune donnée.

Il se compose de trois étapes :

- une source
- des opérations intermédiaires : filter et map…
- une opération terminale qui déclenche le traitement : ex reduce…

Reduce est l'opération d'agrégation essentielle. Des fonctions comme average() renvoient des types, appelées types sécurisés(OptionalDouble), on doit alors traiter le cas d'une collection vide avec une valeur par défaut, ce qui évite les erreurs d'exécution :

```java
List<String> ibans = paiements.stream()                          // source
        .filter( MoyenPaiement m -> m instanceof Virement)       // intermédiaire
        .map( MoyenPaiement m -> ((Virement) m).iban())          // intermédiaire
        .toList();                                               // terminale

int somme = Stream.of( ...values: 1, 2, 3, 4).reduce( identity: 0, Integer::sum);  // 10

double moyenne = paiements.stream() Stream<MoyenPaiement>
        .mapToDouble( MoyenPaiement m -> calculerFrais(m,  montant: 100)) DoubleStream
        .average()          // OptionalDouble
        .orElse( other: 0.0); // défaut si le flux est vide
```

Sources

- https://openjdk.org/jeps/409
- https://openjdk.org/jeps/394
- https://openjdk.org/jeps/441
- https://coddy.tech/learn/fr/java/object_oriented_programming/the_instanceof_operator
- https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/stream/package-summary.html
- https://www.geeksforgeeks.org/java/java-stream-examples/